CC4063: Network Science - Final Project: Comparison of weighted and non-weighted Link Prediction Methods in a Product Rating Dataset

Nina Lichtenberger

ABSTRACT

Link prediction aims to estimate the likelihood of future or missing links in a network based on its observed structure. This study focuses on link prediction using traditional link prediction scores, particularly the role of edge weights, in a bipartite network setting. A set of commonly used structural scores, namely Preferential Attachment, Shortest Path, Local Path, and L3, were computed in both weighted and unweighted forms. These scores were used both independently for ranking candidate links and jointly as input features for supervised classifiers, including Logistic Regression, k-Nearest Neighbors (KNN), and XGBoost. The results show that, among the traditionally ranked scores, Preferential Attachment performed best overall, with no sensitivity to edge weights. Shortest Path showed a minor performance improvement when adjusted for weights. Due to the network's sparsity, Local Path and L3 yielded 0 for all of the negative and for 50% of positive samples, effectively reducing them to binary features. Among the supervised classifiers, XGBoost slightly outperformed the others in 2 of 3 test folds. While XGBoost and Logistic Regression were not influenced by the integration of weights, the KNN performance decreased marginally. Furthermore, feature importance analysis highlighted the relevance of Preferential Attachment and L3 in both weighted and unweighted settings - Local Path only contributed minimally. Feature importance might, however, be influenced by the sparse nature of the network, which makes LP and L3 capture the same information. These findings complement prior research by demonstrating that weighted versions of traditional scores do not have a significant impact on prediction performance in bipartite settings. Limitations related to network sparsity, link prediction methods, and dataset scope suggest future work should explore broader graph types, refined weighting schemes, and more advanced classification and validation techniques.

1. INTRODUCTION

Due to its broad applications in recommender systems, biology, and social network analysis, link prediction is a well-studied problem in network science. The task differs, however, depending on the exact network structure. So far, most work on link prediction has focused on unipartite and unweighted networks. Nonetheless, there is a great number of bipartite and weighted networks in the real world. Bipartite networks comprise, for example, user-product interactions, job seeker–job advertisement interactions, author-paper relationships, and many more. Link prediction in bipartite networks should be considered separately from link prediction in unipartite networks as some of the methods commonly used for unipartite networks are not feasible in bipartite networks [14]. Nonetheless, link prediction in bipartite networks "remains relatively underexplored" [14]. Moreover, networks can be supplemented with additional information as edge weights such as product ratings, number of times the advertisement was clicked, or how many sections an author contributed to a paper. This can be valuable information, which could possibly enhance the performance of link prediction methods. Only a small amount of research has been conducted in the field of implementing edge weights into link prediction mechanisms [4; 1; 17; 12] and almost none has explored these weighted methods for bipartite networks. This paper aims to fill this gap by implementing, comparing, and evaluating the performance of weighted and unweighted versions of link prediction scores on a bipartite network. The explored methods include Preferential Attachment, Local Path, L3, and Path Distance. These are used in a traditional top n ranking approach as well as in the way of input features for supervised classifiers. The paper is structured in the following way: Section 2 reviews the current literature on link prediction in weighted and bipartite networks. Section 3 provides the reader with the necessary theoretical background, elaborating on the definition of the link prediction problem and bipartite networks, giving an overview of link prediction methods, and finally introducing the link scores that were explored in this paper. Section 4 presents the experiments and results. The section starts by giving an overview of the dataset and the sampling process. 4.2 gives the results for the ranking-based, traditional link score classification approach. Section 4.3 exhibits the prediction performance of the classification models as well as the feature importance of the individual scores. Section 5 concludes.

2. LITERATURE REVIEW

The available research has focused on **either** bipartite networks **or** weighted networks.

For bipartite networks, Özer et al compared 19 different methods for three datasets and found that, within traditional link scores, the Structural Perturbation Method, L3, and LP performed best [14]. The Structural Perturbation Method was was not used in this work due to its high computational requirements. They also explored personalized recommendation models, embedding-based link prediction,

and classification models based on topological features. Of the three classifiers they implemented, XGBoost performed best, wherefore XGBoost was also used in this paper. Moreover, recommendation models as well as embeddings led to promising results but are out of scope of this paper. Kunegis et al used the PA index and algebraic link prediction methods on 25 bipartite networks [8]. They observed that the method performance was strongly dependent on the dataset. Other approaches were based on projection [2; 1].

When it comes to weighted networks, Rodrigues de Sá and Prudencio investigated the impact of using weights in link prediction in a co-authorship network [4]. Their results showed that classifiers trained on weighted topological proximity metrics consistently performed slightly better than their counterparts trained on the non-weighted metrics. They were the first ones to adopt weights in a supervised strategy and their results were confirmed by other studies [17]. Also Murata and Moriyasu showed a performance improvement in link prediction in social networks when using the weighted versions of Adamic-Adar, Common Neighbors, and Preferential Attachment [13]. They did, however, observe that the improvement was mainly visible in dense graphs, especially for Preferential Attachment.

In other cases, weights have even been reported to decrease performance [12; 10]. Lü and Zhou attributed this to the weak ties theory which states that the small weight links in social networks play an important role [12; 5]. This might, however, not be true for non-social networks. In social networks, weak ties often act as bridges between communities, facilitating novel information flow. In the case of the product-rating network used for the experiments in this paper, a high weight corresponds to satisfaction with the product, wherefore these products should hold more information about future purchases than low-rated products. This is why the weak ties theory might not hold in a rating network.

This paper aims to contribute to the existing body of literature by applying weighted link scores to a bipartite instead of unipartite network.

3. THEORETICAL BACKGROUND

3.1 Problem Definition

A commonly adopted definition of the link prediction problem is: "Given a snapshot of a network at time t, we seek to accurately predict the edges that will be added to the network during the interval from time t to a given future time t'"[9]. In this paper, the link prediction will concern a bipartite network. Bipartite networks are defined by having their nodes partitioned into two disjoint sets, U and V, such that edges exist only between nodes belonging to different sets. That is, the set of observed edges is given by $L \subseteq U \times V$. Let $L' = (U \times V) \setminus L$ denote the set of all possible but currently missing links in the network. The link prediction task is then formulated as identifying the most probable links within L' that are likely to appear in the network's future evolution. It should be noted that link prediction in bipartite networks requires special handling when compared to unipartite networks [14].

3.2 Overview of Link Prediction Methods

There are three broad distinguishable approaches for link prediction [14]. First, ranking candidate node pairs using heuristic metrics that do not involve a learning process (traditional link scores). Second, using topological features of the network to train a predictive model that estimates link likelihood. Third, applying representation learning to generate graph embeddings, which are then used as inputs for classification instead of handcrafted features. There are also other taxonomies which categorize the methods into based on structural measures or patterns, based on the similarity (content and/or semantics), and based on probabilistic models [4].

This work will focus on traditional link scores because they offer the most straight-forward and easy opportunity to include edge weights. Traditional link scores quantify each potential link $\ell \in L'$ based on structural information derived from the observed graph L [14]. Then, the scores are ranked to identify the most likely candidate links for prediction.

Moreover, we train supervised classifiers to assess the predictive power of feature combinations. The weighted and unweighted link scores will be used as input features. Three classification models are considered: Logistic Regression, k-Nearest Neighbors (KNN), and XGBoost. Each model is trained on the feature set extracted from the training data and subsequently used to predict labels on both the training and test sets. This approach enables a more comprehensive evaluation of the link prediction task, as it leverages the joint distribution of multiple features and allows for non-linear decision boundaries in the case of XGBoost and KNN. The use of classifiers is inspired by Rodrigues de Sá and Prudêncio [4], who compared classifiers trained on the weighted version of Number of Common Neighbors, Jaccard's Coefficient, Preferential Attachment, Adamic-Adar Coefficient, Path Distance, Resource Allocation Index, Local Path, and Local Clustering Coefficient to classifiers trained on the unweighted versions.

3.3 Link Scores

The link scores used in this work are Preferential Attachment (PA), L3, Local Paths (LP), and Path Distance. They were chosen based on the experiments of Özer et al. [14], which showed that LP and L3 yield the best results for bipartite networks when considering performance-computing time ratio. PA was chosen as it can easily be adjusted to consider edge weights and is non-zero as well as meaningful also for node pairs which are not connected or only by longer paths. LP and L3 will only be greater than 0 for node pairs which have relatively short paths (distance of 3). As we will see later, PA has a great advantage for networks with longer distances. All link scores will be computed as a weighted as well as a non-weighted version.

The PA Index describes that nodes with higher degrees are more likely to link to each other [3]. The unweighted formula is given by:

$$\mathrm{PA}(x, y) = k_x \times k_y$$

where k_x and k_y are the degrees of nodes x and y, respectively.

The weighted version is computed as:

$$\mathrm{WPA}(x, y) = s_x \times s_y$$

where

$$s_x = \sum_{z \in \Gamma(x)} w_{xz}$$

is the strength of node x, defined as the sum of weights w_{xz} of edges adjacent to x, and similarly for s_y.

The unweighted version considers the product of node degrees, reflecting the tendency of highly connected nodes to form new links. The weighted version extends this idea by using node strengths, which account for the total weight of connections, thus incorporating the intensity of interactions.

The Local Path index quantifies the likelihood of a link between two nodes by considering the number of common neighbors (paths of length 2) as well as paths of length 3, providing additional structural information. It is defined as [11]:

$$\mathrm{LP}(x, y) = (A^2)_{xy} + \epsilon(A^3)_{xy},$$

where A is the adjacency matrix of the network, and ϵ is a small parameter controlling the contribution of length-3 paths. In bipartite networks, nodes are divided into two disjoint sets with edges only between sets. Paths of even length would connect nodes within the same set, while paths of odd length connect nodes across sets. Since connected pairs of nodes need to belong to different sets, only paths of odd length are possible in bipartite graphs. Therefore, the LP degrades to counting paths of length 3 [14].

The weighted version of the LP replaces binary adjacency values with edge weights, capturing the strength of connections along each path. It is given by:

$$\mathrm{WLP}(x, y) = (W^3)_{xy},$$

where W is the weighted adjacency matrix of the graph, and $(W^3)_{xy}$ aggregates all weighted paths of length 3 between nodes x and y, thereby incorporating both the quantity and intensity of connections.

L3 is very similar. It considers the degree-normalized count of paths of length 3. L3 was introduced by Kovacs et al [7] after they observed that the triadic closure principle was ineffective for certain networks, particularly in biological and bipartite systems. The unweighted L3 index is defined as:

$$\mathrm{L3}(x, y) = \sum_{u,v} \frac{A_{xu} \cdot A_{uv} \cdot A_{vy}}{\sqrt{k_u \cdot k_v}},$$

where A is the binary adjacency matrix and k_u is the degree of node u. The sum runs over all intermediate node pairs (u, v) that form a path of length 3 between nodes x and y.

A weighted version of L3 can be obtained by replacing the binary adjacency values with edge weights w_{ij} and using node strengths $s_i = \sum_j w_{ij}$ iDead of degrees in the normalization. This allows L3 to capture not only presence but also intensity of connections.

Lastly, the Shortest Path Distance (SP) was computed. The SP index for an unweighted networks is defined as the minimum number of nodes that must be traversed from node x to reach node y [16]. A special case occurs when x and y are neighbors, in which case $\mathrm{SP}(x, y) = 1$. In general, a lower SP value indicates a higher likelihood of a link between the two nodes. For weighted networks, the path distance is computed by identifying the minimum weighted path between x and y, where the distance between two adjacent nodes a and b is defined as $\frac{1}{w(a,b)}$, with $w(a, b)$ being the weight of the edge between a and b [4].

4. EXPERIMENTS AND RESULTS

4.1 Dataset and Sampling

The dataset was downloaded from the Koblenz Network Collection [6]. It is a bipartite network of users and products on Amazon.com. Each user connects to all products they have rated. The rating scale ranges from 1 to 5, where 5 denotes the most positive rating. Edge weight denotes the rating score, and edges are timestamped. It has 3,376,972 nodes and 5,743,258 edges. 2,146,057 of the nodes correspond to left nodes (users), whereas the remaining 1,230,915 are products (right nodes).

The timeframe ranges from 31.05.1996 until 29.05.2006. It was decided to use 70% of the edges for training, leading to a split date of 22.01.2005. The training graph contains the edges present before 22.01.2005. The test graph contains the edges appearing after the split date, meaning the 30% of the edges that are the newest. To evaluate the link prediction methods, a balanced set of positive and negative node pairs was sampled from both the training and test graphs. A positive sample is defined as a user-product pair that corresponds to an existing edge in the bipartite graph. A negative sample is a user-product pair that does not exist in the graph (i.e., a non-edge). Specifically, 500 positive and 500 negative pairs were sampled from both the training and the test set. For the positive samples, existing edges between users and products were randomly selected. For the negative samples, user-product pairs were repeatedly generated at random and included only if no edge existed between the user and the product in the graph. This ensures that all negative samples truly represent missing links.

Moreover, the sampling was conducted under the constraint that every node must have at least 3 edges in the training graph. This approach mitigates the cold-start problem in link prediction. This issue arises when nodes with few or no connections lack sufficient information for reliable prediction [15]. Ensuring a minimum degree guarantees that sampled nodes have a baseline level of connectivity, enabling the extraction of meaningful structural features. This not only improves the reliability of feature computation but also enhances model generalization by focusing on nodes embedded in the network structure [15].

To ensure the robustness of results, two additional training and test sets were sampled. They yielded similar results, meaning they showed similar distribution of scores as well as classifier performance. They highest differences between the performance metrics had a magnitude of 0.03 and the exact results for the two additional train-test folds can be found in the appendix.

4.2 Results for Traditional Link Scores

To use the link scores for classification, we adopted two different strategies. The first strategy is the traditional ranking of scores, using a fixed number n for the top scores to be assigned the label 1. The second approach was to find the n that optimizes the accuracy on the training set and use this n for the test set. For each feature under consideration, we began by sorting the training set in descending order according to the feature values. We then identified the optimal number of top-ranked links, n, that maximizes binary classification accuracy by systematically testing all possible n in the sorted training set. Two things should be noted. First, for the SP metric, we only used the fixed n method. Considering that the shortest path for the training pairs will always

be 1 as the training graph contains the train edges, n=500 will yield the highest training accuracy. Second, the scores for the training set should generally be relatively different from the scores of the test set. Both scores are computed on the training graph, but the graph only contains the training and not the test edges. Therefore, it is possible that high accuracy on the training set does not translate into high accuracy on the test set.

When analyzing the distribution of scores, it becomes evident that the network is too sparse for LP and L3 to be meaningful. In the training set, the scores are 0 for 748 pairs and in the test set for 784 pairs. The other train-test folds have similar counts. This essentially means that it does not matter whether the score is weighted or not. Almost all of the nodes with a score (higher than 0) are linked to each other, but this still means that approximately 50% of connected sample pairs are not distinguishable from the non-connected pairs. This also means that LP will achieve the same performance as L3. Considering that it only matters whether there is a path of length 3 or not (non-zero), the degree-normalization does not play a role. When using a fixed n of 500, the two scores achieve an accuracy and F1-score of 0.61 (see Table 1). As can be seen in Table 2, the optimal n slightly differs (253 for LP vs. 243 for L3), leading to an accuracy of 0.68 for LP and 0.67 for L3.

Table 1: Performance Metrics Traditional Scores (n=500)

Feature	n	Train Acc.	Train F1	Test Acc.	Test F1
LP	500	0.6600	0.6600	0.6100	0.6100
PA	500	0.7700	0.7700	0.7400	0.7400
L3	500	0.6480	0.6480	0.6100	0.6100
SP	500	1.0000	1.0000	0.6320	0.6320
LP_w	500	0.6500	0.6500	0.6100	0.6100
PA_w	500	0.7680	0.7680	0.7320	0.7320
L3_w	500	0.6520	0.6520	0.6100	0.6100
SP_w	500	0.9580	0.9580	0.6540	0.6540

Table 2: Performance Metrics Traditional Scores (non-fix n)

Feature	n	Train Acc.	Train F1	Test Acc.	Test F1
LP	253	0.7110	0.6162	0.6790	0.5737
PA	480	0.7760	0.7714	0.7380	0.7327
L3	243	0.7150	0.6164	0.6670	0.5518
LP_w	251	0.7110	0.6152	0.6630	0.5513
PA_w	431	0.7770	0.7605	0.7390	0.7197
L3_w	243	0.7130	0.6137	0.6710	0.5572

The SP index performs slightly better than LP and L3. For a fixed n of 500, it achieves an accuracy and F1-score of 0.63. When including weights, it shows a minor performance improvement to 0.65. This small improvement was also visible in the two other training and test sets. In the additional sets, the inclusion of weights increased the test accuracy by 0.02 and 0.03. When decreasing n, the accuracy improves, whereas the F1-score decreases. A good trade-off can be achieved with a n of 400, leading to 0.67 accuracy and 0.63 F1-score for the weighted version and 0.64 accuracy and 0.60 F1-score for the unweighted version. Hence, we can constitute that including weights slightly improved the SP performance. The improvement is, however, relatively minor. Though proven to be stable across different samples from this dataset, the increase is so small that it might be due to this specific dataset and not visible in every data.

When looking at the distribution of SP values in the test set (Figure 1), we can see that only 40 sample pairs are not connected via any path in the training graph. Nonetheless, the distances are rather high. Only 216 node-pairs are connected via a path of length 3 (shortest path possible for non-neighbors in a bipartite graph) with the majority of them being positive samples. This confirms what we saw earlier with LP and L3 - they are a good indicator of a link if they are existent (non-zero), but this is too seldom in the case of the Amazon Product-Rating dataset. The majority of pairs are connected via a path of 5. There is a high overlap between positive and negative samples when looking at the pairs with SP=5. This explains why the use of SP as a classification score does not yield a higher accuracy than 0.63. For SP higher than 5, the classification becomes easier as more than 75% of samples are negative. For the weighted SP, Figure 2 shows that the overwhelming majority has values between 0.6 and 1.4. Considering that the path lengths are relatively high, this indicates that most edge weights are high (positive ratings), leading to low weighted SP. As low path length and high weights should make links more probable, we can see a concentration of positive samples on the left of the plot. On the right side of the axis, negative samples begin to dominate. Nonetheless, we can also see a high overlap of positive and negative samples in the left center part for weighted SP values of approximately 1.0. All of the observations regarding the score distribution also hold for the two other test folds.

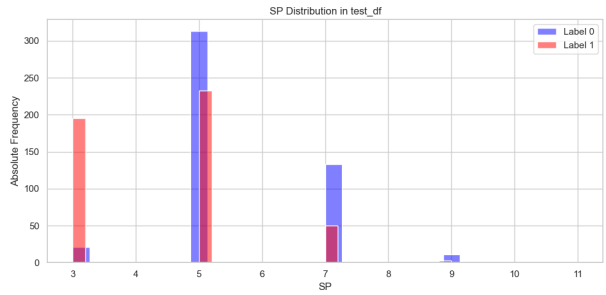

Figure 1: Frequency of SP Values in the Test Set

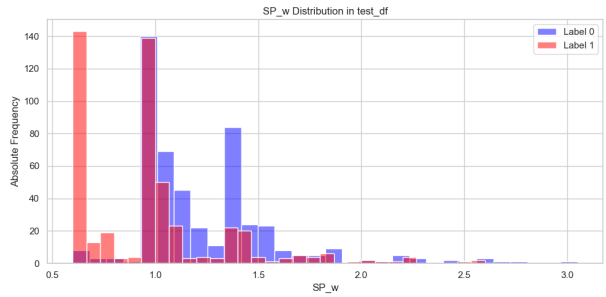

Figure 2: Frequency of weighted SP Values in the Test Set

The best performance is achieved by the PA index. For a fixed n of 500, the unweighted PA achieves an accuracy and F1-score of 0.74. When weighted, the scores decrease to 0.73. This decrease is also visible in the two other train-

test folds (-0.01 and -0.004) but considering the magnitude, not significant. This is in line with the findings of Murata and Moriyasu, who found that performance improvement when including weights was mostly visible in dense networks [13]. As can be seen in the SP values in Figure 1, the Amazon product-rating network is rather sparse, wherefore no performance improvement was expected. Regarding n, the optimal n based on training accuracy is 480 (431 for the weighted version) and, therefore, relatively close to 500. For the other folds, the optimal n for the unweighted PA was determined to be 477 and 429 (weighted 428 and 473, respectively). Despite these differences in optimal n, the training accuracy for the optimal n as well as for n=500 is relatively similar across folds. We can, generally, see a performance decrease when setting n lower than 500. In the training and test set, the density of PA scores for the non-connected nodes is strongly right-skewed (see Figure 3 and 5). This confirms the principle that low-degree nodes are unlikely to connect to each other also holds in this dataset. The same is true for the density of weighted PA scores (Figure 4 and 6). We can, however, observe that the skewness of the distribution decreases. This suggests that weighting smooths out some of the extreme values and brings the non-connected pairs' scores closer to the center. Regarding the PA scores of connected samples, the PA scores seem to be relatively uniformly distributed with some properties of a normal distribution in the training set. In the test set, they clearly resemble a uniform distribution. This might mean that in this dataset, not only high-degree nodes connect, but also mid- and low-degree nodes to some extent. When adjusting for weights, the distribution shifts to a more normal distribution in both cases. This means that the scores are less extreme and more evenly distributed, pulling the shape toward normality. It is also evident that there is a high overlap between the connected and non-connected samples for both the weighted and unweighted PA. All of the observations regarding the score density also hold for the two other train-test folds.

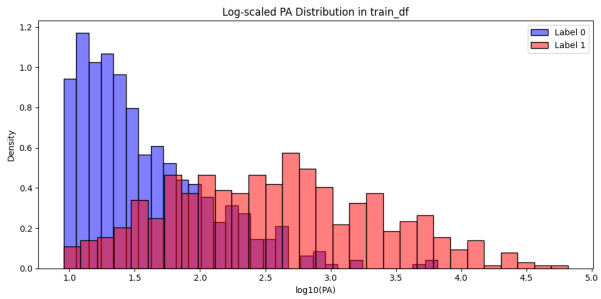

Figure 3: PA Density on a Log-Scale in the Training Set

4.3 Results for Classification Models

4.3.1 Prediction Performance

The link scores were also used as input features for classification models (Logistic Regression, KNN, and XGBoost). The results can be seen in Table 3. The three models were chosen as they have limited assumptions. In contrast to LDA, which was employed by Özer et al [14], for example, the chosen models do not include constraints such as equal

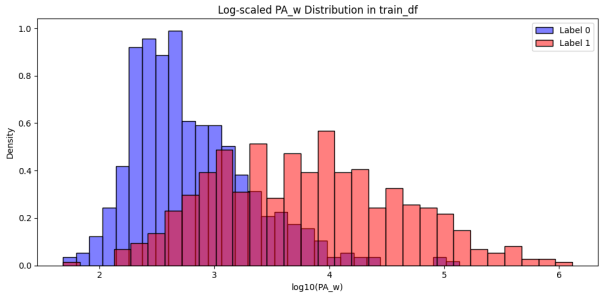

Figure 4: Weighted PA Density on a Log-Scale in the Training Set

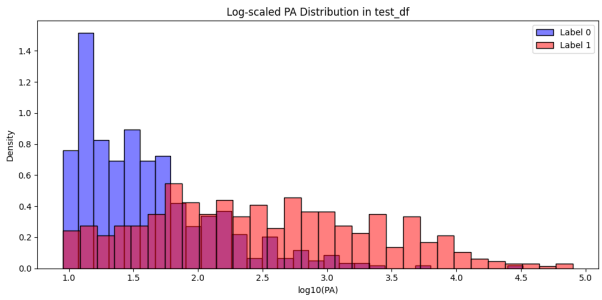

Figure 5: PA Density on a Log-Scale in the Test Set

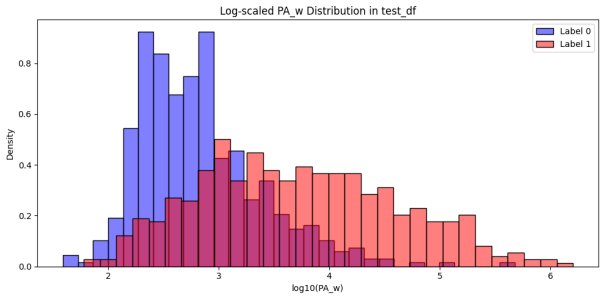

Figure 6: Weighted PA Density on a Log-Scale in the Training Set

Table 3: Model Performance with Different Feature Sets

Feature Set	Model	Train Acc.	Train F1	Test Acc.	Test F1
LP, PA, L3, SP	Logistic Regression	1.0000	1.0000	0.5030	0.0119
LP, PA, L3, SP	KNN	0.9540	0.9557	0.7260	0.6683
LP, PA, L3, SP	XGBoost	1.0000	1.0000	0.5000	0.0000
LP_w, PA_w, L3_w, SP_w	Logistic Regression	0.9640	0.9637	0.6530	0.4829
LP_w, PA_w, L3_w, SP_w	KNN	0.8500	0.8434	0.6860	0.6549
LP_w, PA_w, L3_w, SP_w	XGBoost	0.9900	0.9899	0.6190	0.4129
LP_w, PA_w, L3_w, SP_w, LP, PA, L3, SP	Logistic Regression	1.0000	1.0000	0.5320	0.1203
LP_w, PA_w, L3_w, SP_w, LP, PA, L3, SP	KNN	0.9880	0.9881	0.6760	0.5462
LP_w, PA_w, L3_w, SP_w, LP, PA, L3, SP	XGBoost	0.9990	0.9990	0.5870	0.3012
LP, PA, L3	Logistic Regression	0.7670	0.7155	0.7240	0.6480
LP, PA, L3	KNN	0.7950	0.7902	0.6990	0.6774
LP, PA, L3	XGBoost	0.8400	0.8272	0.7200	0.6868
LP_w, PA_w, L3_w	Logistic Regression	0.7620	0.7032	0.7230	0.6426
LP_w, PA_w, L3_w	KNN	0.8420	0.8358	0.6890	0.6667
LP_w, PA_w, L3_w	XGBoost	0.8520	0.8391	0.7130	0.6843
LP_w, PA_w, L3_w, LP, PA, L3	Logistic Regression	0.7650	0.7102	0.7230	0.6435
LP_w, PA_w, L3_w, LP, PA, L3	KNN	0.8350	0.8254	0.7120	0.6849
LP_w, PA_w, L3_w, LP, PA, L3	XGBoost	0.9190	0.9161	0.7210	0.7103

covariance matrices of the classes or a normal distribution of the features. However, this flexibility comes with trade-offs. KNN and XGBoost are non-parametric and highly flexible models, which can lead to high variance and a greater risk of overfitting, especially on small or imbalanced datasets. Logistic Regression, while more constrained due to its assumption of linearity in the log-odds, exhibits higher bias and less variance. Therefore, further research using additional datasets would be beneficial to evaluate the generalizability and robustness of the results across different network types and data distributions.

When only fed with the unweighted metrics, the Logistic Regression and XGBoost perform very poorly on the test set (equal to random guessing). Both seem to overfit strongly as their accuracy for the training set is 1.0. KNN, in contrast, achieves an accuracy of 0.73 and F1-score of 0.67 on the test set. Nonetheless, it is likely to also overfit as it achieves a training accuracy of 0.95. While the use of weighted measures leads to a slight performance decrease in the KNN model, it improves the performance of the other two models significantly. It should be noted, though, that the impact on KNN performance differed for one of the folds. While two folds saw a small performance decrease, one fold exhibited a small increase in test accuracy. We can, therefore, assume that including weights does neither have a positive nor a negative impact and that the performance differences are due to randomness. The accuracy and F1 of the Logistic Regression and XGBoost, however, increase to 0.65 and 0.48 and 0.62 and 0.41, respectively. Their performance improvement is visible across all folds. When both the unweighted as well as the weighted metrics are used, KNN has an accuracy 0.68, Logistic Regression 0.53, and XGBoost 0.59. Therefore, using unweighted as well as weighted metrics leads to the worst performance for the KNN. This can, however, not be generalized to the other train-test sets. In those, the KNN shows a minor accuracy increase (in comparison to using only the weighted or unweighted features). All of the results above are only true for when all of the scores are used. Superior model performance can be achieved when the SP feature is excluded. This is because in the training set, the unweighted SP is always 1.0 for all positive samples, as the training graph includes all the positive samples of the training set. As a result, the SP feature becomes a nearly perfect predictor for the training data, misleading models such as Logistic Regression and XGBoost to overfit and to rely too strongly on it. In the test set, however, the links to be predicted do not exist in the graph, so that SP is always higher than 1. This mismatch between the feature distribution in training and testing causes poor generalization. Removing SP from the feature set mitigates this overfitting and leads to improved model performance on the test data.

The following observations are for the models trained without SP. When using only unweighted metrics, Logistic Regression performs relatively well with a test accuracy of 0.724 and F1-score of 0.648. KNN shows minimal overfitting, achieving 0.795 accuracy and 0.790 F1 on the training set, while dropping to 0.699 accuracy and 0.677 F1 on the test set. XGBoost exhibits the same slight overfitting, with a test accuracy of 0.720 and F1-score of 0.687. It should be noted that the model ranking depends on the train-test set. In the other folds, Logistic Regression performed worst and XGBoost best. This confirms the research on bipartite networks of Ozer et al [14] who also found that, among the classifiers they tried, XGBoost performed best. When using only the weighted metrics, KNN experiences a small performance drop on the test set (accuracy 0.689, F1-score 0.667), whereas the performance of Logistic Regression and XGBoost remain stable. This was visible across all folds. Combining both unweighted and weighted features only has an impact on the KNN model, which achieves its best performance with all features and a test accuracy of 0.712. As already indicated by the small magnitude of difference, this performance improvement is not stable across folds. In summary, the results are in line with the research of Rodrigues de Sá and Prudencio as well as Zu et al, who found that, depending on the score and dataset, the use of weighted versions as predictor variables either has no impact or a positive impact on predictions of classification models [4; 17].

4.3.2 *Feature Importance*

To gain deeper insight into how the models work, feature importance was computed for each classifier using methods appropriate for the underlying model. For Logistic Regression, importance was derived from the absolute value of the model coefficients, reflecting the strength of each feature's contribution to the decision boundary. For K-Nearest Neighbors (KNN), which is a non-parametric model that does not learn explicit weights, feature importance was estimated using permutation importance, which measures the decrease in model performance (F1-score) when the values of a single feature are randomly shuffled. This captures how much the model relies on each feature for prediction. For XGBoost, importance was computed using the gain metric, which quantifies the average improvement in model performance brought by a feature when it is used to split the data in the decision trees. All importance values were computed using the test set and can be seen in Figure 7 and Figure 8.

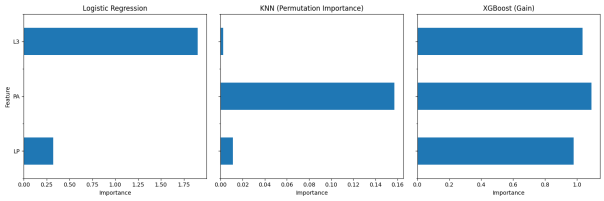

Figure 7: Feature importance comparison using unweighted scores (Feature Set: L3, PA, LP)

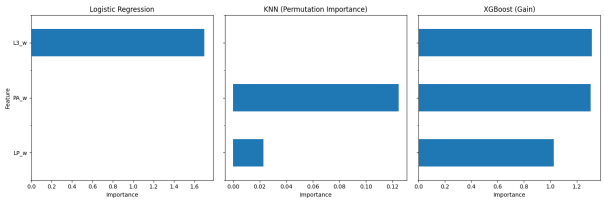

Figure 8: Feature importance comparison using weighted scores (Feature Set: L3_w, PA_w, LP_w)

The feature importance reveals distinct patterns across the three classifiers. For the classifiers using the unweighted features we can observe the following: in Logistic Regression, the L3 index has a substantially higher importance than the scores, suggesting it contributes most strongly to the model's decision boundary in a linear context. PA and LP show minimal influence. This is interesting as we could observe earlier that the network is too sparse to yield meaningful L3 values for most samples due to the high number of pairs with no connecting path of length 3. Nonetheless, Logistic Regression seems to use it as a binary feature (checks if the value is higher than 0). In contrast, KNN's permutation importance indicates PA as the most impactful feature, with LP providing a minor contribution and L3 having almost no effect. XGBoost displays a more balanced reliance across all three features, with all of them showing similarly high gain values. This suggests that XGBoost is effectively leveraging all features in a non-linear fashion, capturing interactions that simpler models may overlook. This did, however, not translate into a strong outperformance. Overall, while each model prioritizes features differently, PA and L3 appear more influential. It makes sense that either L3 or LP (and not both) have a high importance as they essentially capture the same. Considering that the degree-normalization of L3 in comparison to LP does not hold any predictive power in this dataset, it might be due to chance that L3 is assigned higher importance. It should also be noted that both have a high number of 0s for this dataset, which could skew the feature importance. Therefore, it would be necessary to conduct more experiments with other datasets to confirm the results.

When incorporating weights into the link prediction features, the distribution of feature importance across models shifts slightly. In Logistic Regression, the dominance of L3 becomes even more pronounced compared to the unweighted case. The features PA and LP receive near-zero importance. For KNN, the PA feature remains the most influential, similar to the unweighted scenario, but now LP contributes slightly more, while L3 does not even show a low impact anymore. In XGBoost, the importance values for L3 and PA increase and become more balanced, while the relative relevance of LP decreases. In conclusion, we can see that the incorporation of weights influences the relative importance in the way that important features become even more pronounced while non-relevant features lose relevance.

The other train-test folds show similar feature importance. For the unweighted features, the major difference is that LP is assigned a significantly lower importance by the XGBoost model. Other than that, the importance of LP in the Logistic Regression differs depending on the sample. While sample 2 assigns it lower importance relative to L3 than sample 1 (Figure 8), sample 3 assigns it higher importance. Regarding the weighted features, the only visible difference lies in the XGBoost model. In contrast to Figure 8, the other folds show L3 as the clear importance leader, followed by PA. LP is only assigned a relatively low importance (significantly lower than in Figure 8.

5. CONCLUSION

This study examined traditional link prediction scores within a bipartite network, comparing weighted and unweighted variants. The primary contribution lies in the implementation and evaluation of weighted link prediction metrics in a bipartite setting, where such adaptations are less commonly explored. Among the traditional scores, Preferential Attachment consistently performed best across settings, with its performance largely unaffected by the inclusion of edge weights. In contrast, Local Path and L3 indices exhibited limited predictive value, likely due to the sparse nature of the network. For the Shortest Path Score, the inclusion of edge weights provided a minor but consistent improvement across different test sets.

When used as input features for supervised classifiers, the results showed that XGBoost outperformed Logistic Regression and KNN in two of the three evaluated test-sets. While XGBoost and Logistic Regression were not impacted by the inclusion of weights, KNN showed a slight performance degradation under weighted inputs. These findings complement the prior research by Rodrigues de Sá and Prudêncio as well as Zhu et al [4; 17] who found that weighted scores have no impact or a positive impact when used as input features for supervised classification models. Feature importance analysis further highlighted the relevance of PA

and L3 in both weighted and unweighted settings - LP contributed minimally. Feature importance might, however, be influenced by the sparse nature of the network, which makes LP and L3 capture the same information.

Overall, incorporating edge weights only had a minor, insignificant impact on the prediction performance. Despite the valuable insights gained, this study has several limitations that suggest directions for future research. First, the evaluation was restricted to a single bipartite network, which may limit the generalizability of the findings to other domains or graph structures. The sparsity of the network also constrained the applicability of certain scores, such as LP and L3, which rely on the presence of intermediate nodes and paths. Moreover, the exploration of weighting schemes was limited to the direct inclusion of edge weights in existing metrics; alternative weighting strategies or learning-based reweighting could yield different results. Future work could expand the analysis to include more diverse datasets, alternative graph types (e.g., directed networks), and more advanced methods for integrating edge weights (e.g. in representation learning for graph embeddings).

6. REFERENCES

[1] Oussama Allali, Clémence Magnien, and Matthieu Latapy. Link prediction in bipartite graphs using internal links and weighted projection. In *2011 IEEE conference on computer communications workshops (INFOCOM WKSHPS)*, pages 936–941, 2011.

[2] Aslan, S. and Kaya, M. Topic recommendation for authors as a link prediction problem. *Future Generation Computer Systems*, 2018. `https://doi.org/10.1016/j.future.2018.06.050`

[3] Albert-László Barabási, Hawoong Jeong, Zoltán Néda, Erzsebet Ravasz, András Schubert, and Tamás Vicsek. Evolution of the social network of scientific collaborations. *Physica A: Statistical Mechanics and its Applications*, 311:590–614, 2002.

[4] Hially Rodrigues De Sá and Ricardo BC Prudêncio. Supervised link prediction in weighted networks. In *The 2011 international joint conference on neural networks*, pages 2281–2288, 2011.

[5] Granovetter, M. S. The strength of weak ties. *American Journal of Sociology*, 78(6):1360–1380, 1973.

[6] Amazon ratings network dataset – KONECT. `http://konect.cc/networks/amazon-ratings`, October 2017.

[7] I. A. Kovács, K. Luck, K. Spirohn, Y. Wang, C. Pollis, S. Schlabach, W. Bian, D. K. Kim, N. Kishore, T. Hao, M. A. Calderwood, M. Vidal, and A.-L. Barabási. Network-based prediction of protein interactions. *Nature Communications*, 10:1240, 2019.

[8] Kunegis, J., De Luca, E. W., and Albayrak, S The link prediction problem in bipartite networks. In Hüllermeier, E., Kruse, R., and Hoffmann, F. (Eds.), *Computational Intelligence for Knowledge-Based Systems Design*, pages 380–389. Springer, 2010.

[9] David Liben-Nowell and Jon Kleinberg. The link prediction problem for social networks. In *Proceedings of the twelfth international conference on Information and knowledge management*, pages 556–559, 2003.

[10] Liben-Nowell, D. and Kleinberg, J. The link prediction problem for social networks. *Journal of the American Society for Information Science and Technology*, 58(7):1019–1031, 2007.

[11] Linyuan Lü, Chuan-Hong Jin, and Tao Zhou. Similarity index based on local paths for link prediction of complex networks. *Physical Review E*, 80:046122, 2009.

[12] Linyuan Lü and Tao Zhou. Role of weak ties in link prediction of complex networks. In *Proceedings of the 1st ACM international workshop on Complex networks meet information & knowledge management*, pages 55–58, 2009.

[13] Tetsuya Murata and Shingo Moriyasu. Link prediction based on structural properties of online social networks. *New Generation Computing*, 26(3):245–257, 2008.

[14] Şükrü Demir İnan Özer, Günce Keziban Orman, and Vincent Labatut. Link prediction in bipartite networks. In *Procedia Computer Science*, volume 246, pages 2264–2273, 2024.

[15] Andreas I. Schein, Alexandrin Popescul, Lyle H. Ungar, and David M. Pennock. Methods and metrics for cold-start recommendations. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 253–260, 2002.

[16] S. Wasserman and K. Faust. *Social Network Analysis: Methods and Applications*. Cambridge University Press, 1994.

[17] Mingxiang Zhu, Tianjie Cao, and Xiangying Jiang. Using clustering coefficient to construct weighted networks for supervised link prediction. *Social Network Analysis and Mining*, 4:1–8, 2014.

APPENDIX

A. PERFORMANCE METRICS TRADITIONAL SCORES ADDITIONAL SAMPLES

Table 4: Performance Metrics Traditional Scores (n=500) – Sample 2

Feature	n	Train Acc.	Train F1	Test Acc.	Test F1
LP	500	0.6480	0.6480	0.5980	0.5980
PA	500	0.7820	0.7820	0.7320	0.7320
L3	500	0.6480	0.6480	0.6100	0.6100
SP	500	1.0000	1.0000	0.6380	0.6380
LP_w	500	0.6480	0.6480	0.5980	0.5980
PA_w	500	0.7760	0.7760	0.7220	0.7220
L3_w	500	0.6480	0.6480	0.6060	0.6060
SP_w	500	0.9660	0.9660	0.6660	0.6660

Table 5: Performance Metrics Traditional Scores (non-fix n) – Sample 2

Feature	n	Train Acc.	Train F1	Test Acc.	Test F1
LP	263	0.7110	0.6212	0.6810	0.5819
PA	477	0.7890	0.7840	0.7350	0.7288
L3	266	0.7140	0.6266	0.6840	0.5875
LP_w	256	0.7120	0.6190	0.6860	0.5847
PA_w	428	0.7820	0.7651	0.7240	0.7026
L3_w	254	0.7140	0.6207	0.6800	0.5756

Table 6: Performance Metrics Traditional Scores (n=500) – Sample 3

Feature	n	Train Acc.	Train F1	Test Acc.	Test F1
LP	500	0.6380	0.6380	0.5900	0.5900
PA	500	0.7720	0.7720	0.7560	0.7560
L3	500	0.6380	0.6380	0.5900	0.5900
SP	500	1.0000	1.0000	0.6200	0.6200
LP_w	500	0.6380	0.6380	0.5900	0.5900
PA_w	500	0.7760	0.7760	0.7520	0.7520
L3_w	500	0.6380	0.6380	0.5900	0.5900
SP_w	500	0.9620	0.9620	0.6400	0.6400

Table 7: Performance Metrics Traditional Scores (non-fix n) – Sample 3

Feature	n	Train Acc.	Train F1	Test Acc.	Test F1
LP	259	0.7070	0.6140	0.6510	0.5402
PA	429	0.7890	0.7729	0.7510	0.7320
L3	238	0.7120	0.6098	0.6560	0.5339
LP_w	277	0.7090	0.6255	0.6510	0.5508
PA_w	473	0.7870	0.7811	0.7530	0.7461
L3_w	245	0.7110	0.6121	0.6590	0.5423

B. FEATURE IMPORTANCE PLOTS ADDITIONAL SAMPLES

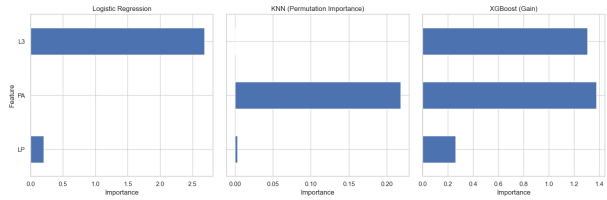

Figure 9: Feature importance unweighted scores (Sample 2)

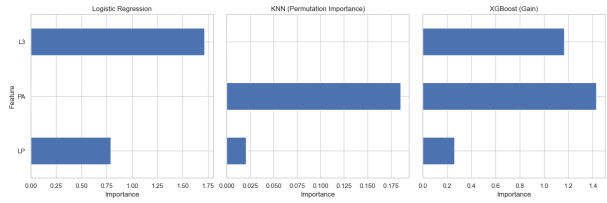

Figure 10: Feature importance unweighted scores (Sample 3)

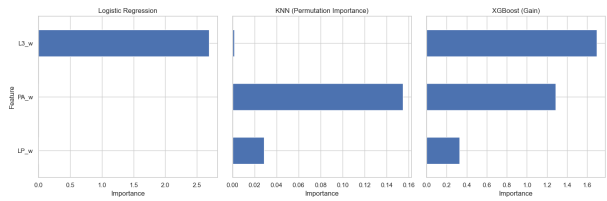

Figure 11: Feature importance weighted scores (Sample 2)

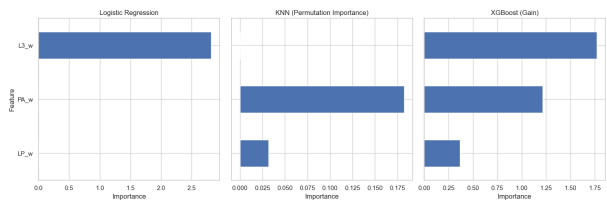

Figure 12: Feature importance weighted scores (Sample 3)

C. PERFORMANCE METRICS CLASSIFICATION MODELS ADDITIONAL SAMPLES

Table 8: Model Performance with Different Feature Sets (Sample 2)

Feature Set	Model	Train Acc.	Train F1	Test Acc.	Test F1
LP, PA, L3, SP	Logistic Regression	1.0000	1.0000	0.5040	0.0159
LP, PA, L3, SP	KNN	0.9570	0.9585	0.7260	0.6667
LP, PA, L3, SP	XGBoost	1.0000	1.0000	0.5000	0.0000
LP_w, PA_w, L3_w, SP_w	Logistic Regression	0.9710	0.9707	0.6570	0.4919
LP_w, PA_w, L3_w, SP_w	KNN	0.8400	0.8323	0.7180	0.6839
LP_w, PA_w, L3_w, SP_w	XGBoost	0.9900	0.9899	0.6070	0.3752
LP, PA, L3, LP_w, PA_w, L3_w, SP, SP_w	Logistic Regression	1.0000	1.0000	0.5510	0.1851
LP, PA, L3, LP_w, PA_w, L3_w, SP, SP_w	KNN	0.8280	0.8189	0.7360	0.7054
LP, PA, L3, LP_w, PA_w, L3_w, SP, SP_w	XGBoost	1.0000	1.0000	0.5000	0.0000
LP, PA, L3	Logistic Regression	0.7560	0.6935	0.7060	0.6101
LP, PA, L3	KNN	0.8200	0.8137	0.7350	0.7196
LP, PA, L3	XGBoost	0.8450	0.8387	0.7230	0.6986
LP_w, PA_w, L3_w	Logistic Regression	0.7570	0.6951	0.7040	0.6022
LP_w, PA_w, L3_w	KNN	0.8210	0.8110	0.7110	0.6848
LP_w, PA_w, L3_w	XGBoost	0.8460	0.8330	0.7200	0.6916
LP_w, PA_w, L3_w, LP, PA, L3	Logistic Regression	0.7630	0.7026	0.7070	0.6057
LP_w, PA_w, L3_w, LP, PA, L3	KNN	0.8210	0.8106	0.7330	0.7088
LP_w, PA_w, L3_w, LP, PA, L3	XGBoost	0.9090	0.9059	0.7190	0.7058

Table 9: Model Performance with Different Feature Sets (Sample 3)

Feature Set	Model	Train Acc.	Train F1	Test Acc.	Test F1
LP, PA, L3, SP	Logistic Regression	1.0000	1.0000	0.5150	0.0583
LP, PA, L3, SP	KNN	0.9690	0.9699	0.7190	0.6577
LP, PA, L3, SP	XGBoost	1.0000	1.0000	0.5000	0.0000
LP_w, PA_w, L3_w, SP_w	Logistic Regression	0.9650	0.9648	0.6340	0.4471
LP_w, PA_w, L3_w, SP_w	KNN	0.8370	0.8318	0.7470	0.7247
LP_w, PA_w, L3_w, SP_w	XGBoost	0.9880	0.9879	0.6260	0.4101
LP, PA, L3, LP_w, PA_w, L3_w, SP, SP_w	Logistic Regression	1.0000	1.0000	0.5240	0.0916
LP, PA, L3, LP_w, PA_w, L3_w, SP, SP_w	KNN	0.8370	0.8332	0.7330	0.7101
LP, PA, L3, LP_w, PA_w, L3_w, SP, SP_w	XGBoost	1.0000	1.0000	0.5000	0.0000
LP, PA, L3	Logistic Regression	0.7440	0.6840	0.7090	0.6255
LP, PA, L3	KNN	0.8190	0.8089	0.7370	0.7055
LP, PA, L3	XGBoost	0.8560	0.8475	0.7470	0.7160
LP_w, PA_w, L3_w	Logistic Regression	0.7450	0.6727	0.7070	0.6088
LP_w, PA_w, L3_w	KNN	0.8260	0.8206	0.7470	0.7288
LP_w, PA_w, L3_w	XGBoost	0.8610	0.8538	0.7390	0.7122
LP_w, PA_w, L3_w, LP, PA, L3	Logistic Regression	0.7550	0.6903	0.7090	0.6115
LP_w, PA_w, L3_w, LP, PA, L3	KNN	0.8260	0.8217	0.7380	0.7177
LP_w, PA_w, L3_w, LP, PA, L3	XGBoost	0.9280	0.9265	0.7330	0.7144